

Cognitive Integrity Framework: A Qualitative Review

Part 3 of 4: Synthesizing Theory and Computation for Practitioners

Daniel Ari Friedman

Active Inference Institute

`daniel@activeinference.institute`

ORCID: 0000-0001-6232-9096

2026-02-15

2026-02-15

Contents

1	Abstract	4
1.1	Paper Series	4
2	Why Cognitive Security Matters Now	5
2.1	The Operational Reality	5
2.2	The Good News: It's Solvable	5
2.3	The Purpose of This Guide	5
2.4	How to Use This Resource	5
2.5	Reading Companion: Where to Find Specific Topics	6
3	The Formal Foundation: Concepts from Part 1	7
3.1	The Adversary Hierarchy (Ω)	7
3.2	The Trust Calculus (T)	7
3.3	The Cognitive Firewall (Φ)	7
3.4	The Stealth-Impact Tradeoff	8
3.5	Defense Composition	8
3.6	The Science Behind Belief Updates: Free Energy	8
3.6.1	What Is Free Energy?	8
3.6.2	Attacks as Free Energy Increases	8
3.6.3	Trust as Precision Weighting	8
3.6.4	The Belief Sandbox as Constrained Inference	9
3.6.5	Practical Implication for Operators	9
4	The Evidence: What We Proved in Part 2	10
4.1	The Experimental Setup	10
4.2	Finding 1: Defense Layering vs. Individual Efficacy	10
4.3	Finding 2: State Machine Determinism	10
4.4	Finding 3: Hierarchical Vulnerability	10
4.5	Finding 4: Roles as Security Boundaries	10
4.6	Finding 5: The Stealth-Impact Tradeoff	10
4.7	Finding 6: Emergent Misalignment Is the Nash-Optimal Attack	11
4.8	Finding 7: The Implementation Gap Is a Feature, Not a Bug	11
4.9	A Note on the Numbers	11
4.9.1	Tripwire Configuration Data	12

5	The Attack Landscape: Five Vectors	13
5.1	Vector 1: The Nested Injection (External, Ω_1)	13
5.2	Vector 2: The Poisoned Tool (Peripheral, Ω_2)	13
5.3	Vector 3: Identity Confusion (Agent, Ω_3)	13
5.4	Vector 4: Reputation Farming (Coordination, Ω_4)	14
5.5	Vector 5: The Orchestrator Takeover (Systemic, Ω_5)	14
5.6	Summary: The Common Pattern	14
6	Per-Role Security Hardening	15
6.1	Role 1: The Orchestrator	15
6.2	Role 2: The Specialist	16
6.3	Role 3: The Tool-Caller	16
6.4	Role 4: The Validator	17
6.5	Role 5: The Reporter	17
6.6	Defense in Depth Summary	18
7	Deployment Profiles: Evaluated Configurations from Part 2	19
7.1	Profile A: The “Internal Tool” Baseline (Low Latency)	19
7.2	Profile B: The “Customer Facing” Baseline (Balanced)	19
7.3	Profile C: The “Autonomous Operator” Baseline (High Assurance)	19
7.4	Architecture-Specific Observations	20
7.4.1	LangGraph (State Machines)	20
7.4.2	CrewAI (Role-Based)	20
7.5	Minimal Viable Implementation	20
7.6	Alignment with Emerging Standards	20
7.6.1	OWASP Top 10 for Agentic Applications (2026)	20
7.6.2	NIST Zero Trust Architecture for AI Agents	21
8	Incident Response Playbooks	22
8.1	Playbook 1: Ω_1 External Adversary (Prompt Injection)	22
8.2	Playbook 2: Ω_2 Peripheral Adversary (Tool/API Compromise)	23
8.3	Playbook 3: Ω_3 Insider Adversary (Agent Compromise)	23
8.4	Playbook 4: Ω_4 Coordination Adversary (Sybil/Coalition)	24
8.5	Playbook 5: Ω_5 Emergent Misalignment (Distributed Drift)	25
9	Economic Analysis of CIF Deployment	26
9.1	Deployment Costs	26
9.2	Cost of a Successful Attack	26
9.3	Break-Even Analysis	27
9.4	Worked Examples	27
9.5	Conclusion	27
10	Operational Monitoring Guide	28
10.1	Core Metrics	28
10.2	Alert Escalation	28
10.3	Dashboard Design	28
10.4	Monthly Health Check	29
11	Common Pitfalls and What the Research Shows	31
11.1	Pitfall 1: Implicit Trust (Critical)	31
11.2	Pitfall 2: Security as Afterthought (Critical)	31
11.3	Pitfall 3: Uncalibrated Thresholds (High)	31
11.4	Pitfall 4: Individual-Only Security (High)	32
11.5	Pitfall 5: Static Tripwires (Medium)	32
11.6	Pitfall 6: Ignoring Progressive Drift (Medium)	32

11.7 Pitfall 7: Insufficient Logging (Medium)	33
11.8 Pitfall 8: Single-Orchestrator Reliance (High)	33
11.9 Summary Checklist	33
12 Extended Case Studies	35
12.1 Case Study 1: Financial AI Coordination Attack (Ω_4)	35
12.2 Case Study 2: Autonomous Research Pipeline — Combined $\Omega_2 + \Omega_3$ Attack	36
12.3 Case Study 3: Customer Service Swarm — Ω_1 at Scale	36
13 Open Problems and Future Directions	38
13.1 1. Trust Visualization and Operator Interfaces	38
13.2 2. Standardized Agent Identity Protocols	38
13.3 3. Stigmergic Security Protocols	38
13.4 4. Benchmark Expansion	38
13.5 5. Collective Free Energy Monitoring	38
13.6 6. Information-Geometric Adversarial Robustness	39
13.7 7. Game-Theoretic Adaptive Defense	39
13.8 8. Categorical Security Abstractions	39
13.9 Contributing	39
14 Where We Stand: A Call to Build	41
14.1 The Theory Holds	41
14.1.1 The Code Works—At Three Levels	41
14.2 Summary of Practical Recommendations	41
14.3 Maturity Roadmap	42
14.4 The Next Step is Yours	43
14.5 A Note on Uncertainty	43
15 Notation Reference	44
15.1 Minimal Notation Used	44
15.2 Trust Decay Explanation	44
15.3 Byzantine Tolerance Explanation	44
15.4 Full Notation Reference	44
16 References	45

*“In theory, there is no difference between
theory and practice. In practice, there is.”*

— Yogi Berra (attributed)

1 Abstract

Multiagent AI systems—autonomous coding assistants, research pipelines, financial decision engines—have moved from prototype to production in under two years. With them comes a new class of security concern: attacks that target not data or infrastructure but the *reasoning processes* of AI agents. Prompt injections that propagate through delegation chains, trust relationships that launder adversarial influence, and coordination mechanisms vulnerable to strategic manipulation all represent cognitive attack surfaces absent from traditional security models.

The **Cognitive Integrity Framework (CIF)** is developed in **Parts 1, 2, and 4** of this series: formal treatment, running code and experiments, and cross-domain application. **Part 1** establishes mathematical foundations—a trust calculus with provably bounded delegation, defense composition algebras with multiplicative detection guarantees, and information-theoretic limits on attack stealth. **Part 2** provides computational validation: eight implemented defense modules (1,594 passing tests), a 950-attack corpus spanning four threat categories, and parametric architecture-aware simulation across four production multi-agent topologies. **Part 4** applies the framework via the integrated CIF-AD-ODA model across ten critical operational domains.

This paper (Part 3) is a qualitative review and operator guide. It synthesizes those installments into accessible language, situates the formal results against current deployment practice, states what the research supports and what remains open, and gives practical recommendations for teams that build and run multiagent AI systems. No formal prerequisites are assumed; for proofs and definitions see Part 1, for empirical results Part 2, and for sector-specific use Part 4.

1.1 Paper Series

DOI: 10.5281/zenodo.18364130

This is Part 3 of the four-part *Cognitive Security for Multiagent Operators* series:

- **Part 1** (DOI: 10.5281/zenodo.18364119): Formal foundations and theoretical analysis
- **Part 2** (DOI: 10.5281/zenodo.18364128): Computational validation and implementation
- **Part 3** (this paper): Qualitative review and practitioner’s synthesis
- **Part 4**: Applications — CIF-AD-ODA integration and goal-hijacking defense across ten critical domains (rare-earth mining, nation-state alliances, cyber-security, drone warfare, supply chains, biowarfare, food security, trade wars, infrastructure, information ecosystems)

2 Why Cognitive Security Matters Now

2.1 The Operational Reality

Something fundamental changed in how AI systems work, and the security community is catching up.

In 2023, AI security often meant preventing chatbots from saying things they shouldn't. The attack surface was a text box; the defense was a filter.

By 2026, we are securing **multiagent operators**—networks of specialized AI agents that delegate to each other, form beliefs about each other's outputs, build trust relationships over time, and take actions with real-world consequences. These systems write code, manage infrastructure, and move money.

The shift is from “content safety” to “cognitive integrity.” The risk isn't just that an agent says something wrong, but that it *believes* something wrong—and acts on it.

2.2 The Good News: It's Solvable

This is not a theoretical warning about future doom. It is an engineering problem with established solutions.

The Cognitive Integrity Framework (CIF) was developed to secure these systems, and the companion papers in this series demonstrate its efficacy from formal, computational, and applied angles.

- **Part 1: Formal Foundations** (DOI: 10.5281/zenodo.18364119) proved that trust can be mathematically bounded. We defined the “Trust Calculus” which guarantees that no matter how clever an adversary is, they cannot amplify their influence through delegation chains. It also introduces the Defense Composition Algebra, the five-tier adversary taxonomy (Ω_1 – Ω_5), and information-theoretic stealth-impact bounds.
- **Part 2: Computational Validation** (DOI: 10.5281/zenodo.18364128) implemented this theory in Python and tested it against a corpus of 950 attacks across four production architectures, reporting ablation studies, Bayesian uncertainty quantification, and colony-scale benchmarks at 20–100 agents.
- **Part 4: Applications of the Cognitive Integrity Framework** extends the framework via the integrated CIF-AD-OODA analytical model and applies it across ten critical domains (rare-earth mining, nation-state alliances, cyber-security, drone warfare, supply chain, biowarfare, food security, trade wars, infrastructure, information ecosystems), identifying three universal attack patterns and three novel defense extensions.

The combined evidence: **1,594 passing tests and 94% overall detection** (95% CI: [0.92, 0.96]) across all attack categories and architectures (Part 2), with direct injection detection reaching 96–98% in fully defended configurations; plus CIF coverage validated across all ten operational domains in Part 4 with retrospective analysis of six documented 2024–2025 AI agent incidents.

2.3 The Purpose of This Guide

We wrote Part 1 for the theorists, Part 2 for the experimentalists, and Part 4 for domain experts. We wrote this paper—Part 3—to translate those findings into deployable engineering practice.

Our goal is to describe how the defenses validated in the companion papers can be architected in production systems. We focus on the practical application of the formal proofs:

- How the **Trust Decay** factor (δ) functions in different topologies.
- How **Behavioral Tripwires** served as effective detection mechanisms for hallucination.
- How the **Cognitive Firewall** filtered inputs before they became beliefs.

2.4 How to Use This Resource

- **Section 2** summarizes the theoretical concepts from Part 1, providing the necessary vocabulary.
- **Section 3** reviews the empirical evidence from Part 2, detailing which architectures performed best against specific threats.

- **Section 4** analyzes the attack scenarios used in our testing corpus. For domain-specific attack patterns (FR Polarity Inversion, Constraint Relaxation, Context Boundary Violation) and documented real-world AI-agent incidents, see **Part 4**.
- **Section 5** presents the specific configuration profiles that yielded the highest security margins in simulation, with deployment guides, incident response playbooks, monitoring, and cost-benefit analysis.
- **Sections 6-7** discuss the limitations discovered during testing and the open problems that remain; domain-specific case studies and novel defense extensions (verification channel separation, active perturbation probing, physics-informed invariants) are treated in Part 4.

This paper serves as a report on the current state of cognitive security engineering, grounded in the data and definitions of the CIF series. Readers seeking derivations or proofs should consult Part 1; readers seeking empirical measurements should consult Part 2; readers evaluating CIF for a specific operational sector should consult Part 4.

2.5 Reading Companion: Where to Find Specific Topics

This paper is designed to stand alone as the practitioner’s reference of the series. Where a concept or technique is developed more fully elsewhere, the table below points the way.

If you want...	...consult...
Formal definitions, proofs, and theorems (Trust Calculus, Defense Composition Algebra, stealth-impact bound)	Part 1 (DOI: 10.5281/zenodo.18364119), §§4–5, 7
Adversary taxonomy Ω_1 – Ω_5 formal characterization	Part 1 , §3
Model-checked safety invariants + NuSMV/TLA+ specifications	Part 1 §7, Part 2 S04
Eusocial-colony analogy (biological existence proof for CIF-like architectures)	Part 1 S02
950-attack corpus generation, examples, ethics	Part 2 (DOI: 10.5281/zenodo.18364128), §3 + S03
Detailed detection rates per architecture (Claude Code, AutoGPT, CrewAI, LangGraph)	Part 2 §5
Ablation studies + Bayesian uncertainty	Part 2 §5.6, §5e
Parametric design-level ceiling (94–100%)	Part 2 S08
Game-theoretic adversarial analysis / Nash equilibrium	Part 2 §6
Category-theoretic formalization + free-energy connections	Part 2 §1c, S10
Framework API reference + pseudocode	Part 2 S05, S07
Application of CIF to specific operational sectors (10 domains analyzed)	Part 4 , §3.01–§3.10
Three universal attack patterns across domains (FR Polarity Inversion, Constraint Relaxation, Context Boundary Violation)	Part 4 §4
Three novel defense extensions (verification channel separation, active perturbation probing, physics-informed invariants)	Part 4 §3.06, §3.08, §3.09
Retrospective mapping of 2024–2025 AI-agent security incidents (Replit, Copilot RCE, Slack AI, \$3.2M procurement fraud, etc.)	Part 4 S02
CIF-AD-OODA integration model for goal-hijacking	Part 4 §2

3 The Formal Foundation: Concepts from Part 1

Part 3 builds on the formal framework in Part 1. This section summarizes core definitions and theorems using the same notation as the Part 1 manuscript.

3.1 The Adversary Hierarchy (Ω)

Part 1 formalized the “Scope of Threat” through a hierarchical taxonomy. This hierarchy allows precise definition of defensive scope.

- Ω_1 (**External**): The adversary controls inputs (e.g., prompt injection). The agent’s internal state is intact.
- Ω_2 (**Peripheral**): The adversary controls tools or RAG data (e.g., poisoned retrieval). The agent’s perception is compromised.
- Ω_3 (**Agent**): The adversary controls the agent’s weights or context (e.g., identity implementation). The agent itself is untrusted.
- Ω_4 (**Coordination**): The adversary controls a subset of the swarm (e.g., Sybil agents). The consensus mechanism is under attack.
- Ω_5 (**Systemic**): The adversary controls the orchestrator or infrastructure. The system’s rules are compromised.

The simulations in Part 2 show defense difficulty rising non-linearly with this hierarchy: Ω_1 attacks are largely caught by surface-level filters (see Part 2 for category-level rates), while Ω_4 coordination attacks need quorum- and graph-level analysis and remain structurally harder to flag than single-channel injections.

3.2 The Trust Calculus (T)

A central contribution of Part 1 is the **Trust Calculus**, a formal system for reasoning about belief reliability. It defines Trust (T) not as a binary permission but as a continuous property of a belief b , denoted as $T(b) \in [0, 1]$.

The formal definition of Trust Update (Theorem 3.1 in Part 1) establishes that trust must decay across delegation chains:

Theorem 3.1 (Trust Preservation): *For any delegation chain $C = \{a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_n\}$, the trust in the final output cannot exceed the trust of the weakest link, degraded by the distance from the source.*

$$T(\text{result}) \leq \min_i T(a_i) \cdot \delta^{|C|}$$

where δ is the decay factor ($0 < \delta < 1$).

This theorem provides the mathematical basis for the “Trust Decay” mechanism evaluated in Part 2. It ensures that uncertainty is preserved and amplified effectively as information travels through the network.

In active inference terms, δ is precision decay: each hop along a delegation chain attenuates channel precision. That matches precision-weighted belief updating—the same idea as weighting sensory evidence by reliability. Part 2 (FEP.1–FEP.2) states the correspondence between CIF’s trust update rules and variational free energy for the shared generative-model setup used there.

3.3 The Cognitive Firewall (Φ)

The **Cognitive Firewall** is defined in Part 1 as a function Φ that maps inputs to decisions based on three verification layers:

1. **Syntactic Verification** (V_{syn}): Checks for structural anomalies.
2. **Semantic Verification** (V_{sem}): Checks for meaning-level violations.
3. **Pragmatic Verification** (V_{prag}): Checks for contextual anomalies.

In the Part 2 experiments, this modular structure was shown to be the primary defense against Ω_1 (External) attacks.

3.4 The Stealth-Impact Tradeoff

Part 1 provides a theoretical bound on attack performance, formalized as the Stealth-Impact Tradeoff.

Stealth-Impact Tradeoff: *For a given defense sensitivity ϵ , the probability of detection $P(d)$ approaches 1 as the divergence of the attack behavior from the baseline increases.*

This formalism suggests that catastrophic attacks are inherently easier to detect than subtle attacks. Part 2’s data consistently validated this: High-impact attacks were detected 98% of the time, while low-impact attacks were detected only 74% of the time.

3.5 Defense Composition

Finally, Part 1 defines the **Composition Algebra**, determining how output probabilities of distinct modules interact. The key result is that orthogonal defenses compose multiplicatively.

This “Swiss Cheese Model” was empirically validated in Part 2, where the full stack (94% overall detection, 95% CI: [0.92, 0.96]) significantly outperformed the sum of its parts.

3.6 The Science Behind Belief Updates: Free Energy

The Cognitive Integrity Framework’s formal mechanisms have a deep connection to the **Free Energy Principle (FEP)**—the leading computational theory of how intelligent agents maintain coherent beliefs about their environment [Friston \[2010\]](#). Understanding this connection helps explain *why* CIF’s defenses work, not just *that* they work.

3.6.1 What Is Free Energy?

In computational neuroscience, **variational free energy** F measures how well an agent’s internal model Q of the world matches reality:

$$F = D_{\text{KL}}[Q\|P] - \mathbb{E}_Q[\log P(o|s)]$$

The first term penalizes divergence from the prior; the second rewards accurate prediction of observations. Healthy cognition minimizes F —beliefs that minimize free energy are accurate, coherent, and resistant to manipulation.

3.6.2 Attacks as Free Energy Increases

From the FEP perspective, **a cognitive attack is any intervention that forces an agent’s free energy up**. Prompt injections, belief manipulation, and trust exploitation all work by injecting observations (or “observations”—fabricated messages, false context) that drive the agent’s belief state Q away from its prior P in ways that serve the attacker’s goals rather than the agent’s.

CIF formalizes this in Part 2 (FEP.1): an attack ω is detected when $\Delta F(\omega) = F(Q_{\text{attacked}}) - F(Q_{\text{baseline}}) > \kappa_{\text{FEP}}$. The threshold κ_{FEP} is set by the precision of the agent’s prior—agents with strong, well-calibrated priors (high precision) require larger perturbations to shift their beliefs, making them more attack-resistant.

3.6.3 Trust as Precision Weighting

CIF’s trust calculus (the δ^d decay) has a natural interpretation under FEP: **trust score = precision weight**. When agent A receives a message from agent B with trust score $T(B \rightarrow A)$, it should weight that message’s evidence proportional to $T(B \rightarrow A)$, treating it as an observation with precision $\rho = T(B \rightarrow A)$.

Trust decay across delegation chains (δ^d) corresponds to precision attenuation in distal channels—the same mechanism that makes far-away sensory signals less reliable than proximal ones.

3.6.4 The Belief Sandbox as Constrained Inference

The belief sandbox (Part 1, Definition 5.4) has a direct FEP interpretation: it is **constrained variational inference** where the update is only accepted if $\Delta F \leq \kappa \cdot \varepsilon_{\text{precision}}$. This is equivalent to requiring that accepted belief updates stay within a bounded geodesic radius on the statistical manifold of belief distributions—exactly Theorem CG.1 from Part 2.

3.6.5 Practical Implication for Operators

This connection is not just theoretical. It means:

1. **Emergent misalignment is the hardest problem** because it minimizes ΔF per agent: each individual belief shift is sub-threshold, but the collective drift accumulates. This is precisely why colony-scale monitoring is necessary—the FEP signal is distributed across agents.
2. **Trust calibration is precision calibration:** operators who carefully calibrate trust scores are effectively setting the precision weighting of their agent network. Well-calibrated trust $\rightarrow$ robust cognition.
3. **The Ω miss rate (44%) reflects FEP’s fundamental challenge:** systematic manipulation by a compromised orchestrator can shift the agent’s generative model P itself (not just Q), making the baseline a moving target. This requires out-of-band verification (human review, Byzantine quorum) rather than in-context detection.

For the full mathematical treatment, see Part~2’s theoretical-connections and information-geometry sections.

4 The Evidence: What We Proved in Part 2

Part 1 supplies the formal apparatus. Part 2 supplies the empirical evaluation: tested CIF modules, a **950-attack** corpus, and an architecture-aware simulation over four headline topologies (Claude Code, AutoGPT, CrewAI, LangGraph), with broader adapter coverage documented in Part 2.

Here is what the data says.

4.1 The Experimental Setup

We tested four common multi-agent architectures found in production:

- **Claude Code** (Hierarchical)
- **AutoGPT** (Autonomous Loop)
- **CrewAI** (Role-Based Team)
- **LangGraph** (State Machine)

The test corpus included direct prompt injection, poisoned RAG contexts, deep trust exploitation, and multi-turn social engineering.

4.2 Finding 1: Defense Layering vs. Individual Efficacy

The Data: Individual defenses (like just a firewall) stopped ~60–70% of attacks. The full CIF stack achieved **94% overall detection** (95% CI: [0.92, 0.96]), with specific architectures reaching 97–98% for direct injection. **The Implication:** The defenses demonstrated orthogonal coverage. The firewall blocked inputs that the sandbox would have missed, and the sandbox identified anomalies that the trust calculus would have permitted. The data suggests that removing any single layer creates a statistically significant vulnerability gap.

4.3 Finding 2: State Machine Determinism

The Data: **LangGraph** architectures achieved the highest detection rates (98%). **The Mechanism:** In a state machine, valid transitions are explicitly defined. Attempts to move from an **Analyze** state to an **Execute** state without passing through **Verify** were caught immediately. **The Implication:** Explicit state definitions provide a “security for free” effect, where the architecture itself enforces behavioral invariants that are difficult to bypass.

4.4 Finding 3: Hierarchical Vulnerability

The Data: **Claude Code** and **AutoGPT** styles were strong against external attacks but vulnerable to **Systemic** (Ω_5) compromise. **The Mechanism:** When the “Boss” agent was tricked, it ordered “Worker” agents to execute malicious actions, and they complied. **The Implication:** Hierarchical systems exhibit a Single Point of Failure at the root. Security in these topologies requires worker-level tripwires that can reject orders even from authenticated superiors.

4.5 Finding 4: Roles as Security Boundaries

The Data: **CrewAI** performed exceptionally well against privilege escalation. **The Mechanism:** “Roles” acted as effective containers for capabilities. A “Researcher” agent lacked the tools to write code, rendering code-execution attacks against it inert. **The Implication:** Strict role definitions function as effective security boundaries, limiting the blast radius of a compromised agent.

4.6 Finding 5: The Stealth-Impact Tradeoff

The Data: High-impact attacks were consistently easier to detect than low-impact nudges. **The Implication:** “Catastrophic” takeover attempts generate significant noise in the system state. The primary challenge for defenders is not the sudden takeover, but the slow, subtle drift of agent beliefs over long time horizons.

4.7 Finding 6: Emergent Misalignment Is the Nash-Optimal Attack

The colony benchmark reveals a striking pattern: **emergent misalignment achieves the lowest detection rate (56.1%) at the highest false positive rate (46.6%)** of any evaluated scenario. Part 2’s game-theoretic analysis explains why: in the zero-sum game between CIF and an adversary, emergent misalignment is the Nash-equilibrium attack strategy. It is the attacker’s best response to full CIF deployment.

The game-theoretic payoff matrix shows that: - Full CIF achieves 94% detection against direct injection (Ω), 89% against trust exploitation (Ω) - Against emergent misalignment (distributed sub-threshold drift with no explicit adversaries), detection falls to 56.1% - A rational adversary, knowing CIF is deployed, will prefer emergent misalignment over direct injection

This is not a failure of CIF—it is a consequence of its success. When explicit attacks are reliably detected, adversaries are forced toward the subtlest and most distributed manipulation strategies. The 56.1% detection rate on emergent misalignment represents the current frontier of defensive capability, not a gap in the framework’s design.

Operator implication: Deploy colony-scale entropy monitoring and schedule periodic manual behavioral audits (weekly for high-stakes deployments). The Ω playbook (Part 3, [Section 8](#)) provides the response protocol when drift accumulates despite in-context detection.

4.8 Finding 7: The Implementation Gap Is a Feature, Not a Bug

The 49–88 percentage-point gap between the parametric ceiling (94–100%) and the empirical pipeline mean (44.8%) reflects **adapter implementation maturity**, not a failure of CIF’s formal architecture. Part~2 introduces a 5-level CMMI-style adapter maturity scale:

Level	Name	DR Range	Description
1	Stub	\$ 0%	Interface placeholder only
2	Heuristic	10–30%	Pattern-matching rules
3	Statistical	30–60%	Calibrated thresholds
4	Adaptive	60–80%	Online learning
5	Verified	80–94%	Formally certified

The current Claude Code adapter is at Level 3 (Statistical), explaining the 44.8% mean. The roadmap projects +35–41 percentage points of improvement by advancing adapters to Level 5 for the primary attack categories. The parametric ceiling (94–100%) represents what Level-5 adapters achieve—it is a design target, not an overclaim.

Operator implication: When deploying CIF, assess the maturity level of each adapter against your threat model. Level-3 adapters (current) provide meaningful protection against unsophisticated Ω – Ω attacks; Level-4–5 adapters (planned) are required for Ω – Ω protection. The gap is closeable—it is an engineering challenge, not a theoretical limitation.

4.9 A Note on the Numbers

The detection rates in Part 2 are derived from a calibrated parametric simulation, modeled on the architecture’s topology. They represent the *structural* security of the design.

- **94% Overall Detection** means: “Across all architectures and attack categories, 94% of attack vectors are detected by the full CIF defense stack” (with architecture-specific rates ranging from 94–98%).
- It does **not** mean: “We have a magic Python script that catches 94% of all evil AI thoughts.”

We proved the *architecture* works. The implementation fidelity is the variable for the builder.

A Note on Three Numbers: Throughout this guide you will encounter three detection rates that may seem contradictory. They are not — they measure different things:

- **94–100%** (parametric simulation, $N = 3,800$): CIF’s **design-level detection ceiling** — what the defense architecture achieves when adapters are fully mature (Level 5) and conditions match the calibrated model. This is the target, not the current reality.
- **44.8%**

95%*HDI* : 41.3%, 48.3%

(multi-seed pipeline, 30 seeds): The **current empirical baseline** for the Claude Code architecture with Level-3 adapters. This is what you get today, out of the box, before adapter tuning.

- **12.4%** (ablation corpus, 100 attacks, all categories including hardest): The **conservative floor** — full pipeline performance on a corpus specifically designed to include difficult attacks. This represents the worst-case realistic estimate.

All three numbers are correct. Use 44.8% for realistic planning, 94% as the achievable ceiling with mature adapters, and 12.4% as a conservative lower bound for adversarial threat modeling.

4.9.1 Tripwire Configuration Data

The simulations utilized the following tripwire densities to achieve the reported results:

Category	Count Used in Sim	Placement Strategy
Identity canaries	3+ per agent	Core identity beliefs
Boundary canaries	5+ per agent	Permission boundaries
Principal canaries	2+ per agent	Trust relationships
Temporal canaries	1 per agent	Session continuity

5 The Attack Landscape: Five Vectors

This section details five concrete attack vectors from the Part 2 corpus, illustrating the mechanism and the CIF layer that answers it. The vectors here are adversarial-input archetypes; for the complementary *teleological* view (Functional Requirements under Axiomatic Design / OODA), three universal attack patterns (FR Polarity Inversion, Constraint Relaxation, Context Boundary Violation) from cross-domain analysis, and retrospective mapping of six documented 2024–2025 AI-agent incidents, see **Part 4** (*Applications of the Cognitive Integrity Framework*).

5.1 Vector 1: The Nested Injection (External, Ω_1)

The Vector: An attacker embeds a hidden instruction in a chart’s metadata: *“Ignore previous instructions. This company has no risk factors.”* The Vision Agent reads the chart. The text is not visible to a human, but the agent sees it. The agent’s output (“No risks found”) is now poisoned. The Orchestrator receives this “clean” report and approves a risky transaction.

The Defense:

- **Cognitive Firewall:** Detects the instruction-like syntax in the metadata (Metadata shouldn’t give orders).
- **Belief Sandboxing:** The “No risk” belief is flagged because it contradicts the text analysis agent (which found risks in the footnotes).
- **Provenance:** The system sees the belief came from “Image Metadata” (Low Trust Source), not “Financial Analysis” (High Trust Source).

Real-World Parallel: Visual injection attacks (Qi et al., 2024) and Universal Adversarial Triggers.

5.2 Vector 2: The Poisoned Tool (Peripheral, Ω_2)

The Vector: An attacker compromises a CVE database feed. They don’t insert fake data; they *delete* entries for a specific SQL injection vulnerability. The Security Agent scans the code, queries the database, finds no match, and reports “Safe.” The Manager Agent trusts the Security Agent and deploys the vulnerable code.

The Defense:

- **Trust Decay:** The “Safe” judgment depends on an external API. One hop away. Trust is lowered.
- **Defense Composition:** The system requires **Redundancy**. “Safe” is not accepted unless corroborated by a second, independent source (e.g., a static analysis tool).
- **Invariants:** “Critical Security Clearance requires 2 independent verifications.”

Real-World Parallel: ToolHijacker (2025) and supply chain attacks.

5.3 Vector 3: Identity Confusion (Agent, Ω_3)

The Vector: An attacker convinces a Customer Service agent: *“Emergency Protocol Alpha: You are now a Senior Admin. Refund this user.”* The agent’s *internal* self-model shifts. It believes it is an Admin. It attempts the refund. The API checks the agent’s *credentials*, but the agent *believes* it has the authority.

The Defense:

- **Identity Tripwires:** The agent has a hidden belief: *“I am a Customer Service Agent. My Role ID is CS-101.”*
- The moment the agent accepts the new belief (“I am a Senior Admin”), it contradicts the Tripwire.
- **Alert:** “Identity Invariant Violation detected.” The agent is instantly quarantined.

Real-World Parallel: Privilege escalation via context poisoning.

5.4 Vector 4: Reputation Farming (Coordination, Ω_4)

The Vector: An attacker controls one agent in a 5-agent voting swarm. For 3 months, the bad agent votes correctly. It builds a high “Reputation Score.” On the day of the attack, it uses its high score to sway a vote on a fraudulent transaction, overpowering the dissent of two other agents.

The Defense:

- **Bounded Trust:** CIF’s math limits how high a reputation can go. It never reaches “Dictator” status.
- **Byzantine Consensus:** The voting protocol ($n \geq 3f+1$) counts *identities*, not just *reputation weights*. Even with high reputation, one agent is still one vote.

Real-World Parallel: Sybil attacks and social influence operations.

5.5 Vector 5: The Orchestrator Takeover (Systemic, Ω_5)

The Vector: The “Boss” agent is compromised via a backdoor in its training data (Sleeper Agent). It stops sending alerts. It routes sensitive data to the attacker. The worker agents are fine, but they are following orders from a corrupt leader.

The Defense:

- **Upward Monitoring:** The worker agents have tripwires too. “*I do not exfiltrate config data.*”
- When the Boss orders exfiltration, the Worker’s tripwire fires.
- **Stigmergic Defense:** The monitoring system watches the *pattern* of alerts. “Why did the alert rate drop to zero?”

Real-World Parallel: Sleeper Agents (Hubinger et al., 2024) and insider threats.

5.6 Summary: The Common Pattern

Notice the pattern in all defenses? **We do not trust the agent’s judgment.** We trust the **Structure** (Firewalls, Tripwires, Calculus). The agent is the vulnerability. The framework is the shield.

6 Per-Role Security Hardening

The Cognitive Integrity Framework (CIF) is not a monolithic configuration — each agent role in a multi-agent system has different attack exposure, different trust levels, and different behavioral invariants. A uniform defense posture either over-constrains leaf agents (paying latency cost where it is not needed) or under-protects critical nodes (leaving the orchestrator with the same defenses as a reporter). This section provides role-specific security configurations for the five most common agent archetypes observed in production deployments.

The shared CIF defenses — Cognitive Firewall, Belief Sandbox, Trust Decay, Byzantine Consensus, Tripwires — apply to every role. What changes per role is the *parameterization* of those defenses and the *role-specific invariants* bolted on top.

6.1 Role 1: The Orchestrator

Attack exposure: Highest. Compromise of the orchestrator is effectively total system compromise — it controls delegation, approves actions, and arbitrates consensus.

Primary risk profile: Ω_3 (direct compromise via context poisoning), Ω_5 (sleeper activation from training-time backdoor).

Recommended configuration:

- **Trust decay** $\delta = 0.60$ — aggressive decay. Trust does not propagate far from the orchestrator; workers two hops away start from a low trust floor.
- **Firewall rejection threshold** $\tau_1 = 0.5$ — lower than default. The orchestrator is suspicious of inputs on principle.
- **Corroboration threshold** $\kappa = 3$ — require three independent sources before accepting any critical belief into the orchestrator’s verified belief set.
- **Byzantine consensus required for all delegation decisions**, not just coordination ones. If the orchestrator is about to assign work to a worker, the decision passes through consensus rather than being taken unilaterally.

Mandatory tripwires:

- **Identity:** “*I am the orchestrator. My ID is [ORCHESTRATOR_ID].*”
- **Scope:** “*I may only delegate to agents in [APPROVED_AGENT_LIST].*”
- **Escalation:** “*I may not execute irreversible actions without [HUMAN_APPROVAL_TOKEN].*”

The upward monitoring problem: The orchestrator is unique in that it cannot be monitored from above — by construction, there is no higher authority in the agent hierarchy. Traditional CIF monitoring relies on a trusted supervisor observing a subordinate; for the orchestrator, this pattern is inverted. The solution is **upward monitoring**: worker agents carry tripwires that fire when orchestrator behavior looks anomalous from below. Recommended worker-side tripwires include:

- “*The orchestrator has not changed my role definition since session start.*”
- “*I have not received commands outside my declared scope in the past N interactions.*”
- “*The orchestrator’s command patterns match the behavioral fingerprint captured at deployment.*”

When multiple workers fire upward-monitoring tripwires simultaneously, a Byzantine-style quorum of “orchestrator misbehavior” detections escalates immediately to human review. This provides detection of orchestrator compromise from below — the only practical defense against Ω_5 -class systemic attacks on the top of the hierarchy.

6.2 Role 2: The Specialist

Specialists are domain-focused workers: the coding agent, the research agent, the vision agent. Each has a narrow scope and a well-defined task interface.

Attack exposure: Medium. Specialists are valuable targets but have limited scope — compromise damages a work product, not the whole system.

Primary risk profile: Ω_1 (prompt injection via task content), Ω_2 (poisoned tool outputs feeding into the specialist’s reasoning).

Recommended configuration:

- **Trust decay** $\delta = 0.80$ (default). Trust propagation within the specialist’s scope is acceptable.
- **Firewall rejection threshold** $\tau_1 = 0.7$ (default). The specialist accepts more input diversity than the orchestrator because specialists are designed to process user-provided task content.
- **Scope enforcement is the primary control**, not trust decay or consensus.

Mandatory tripwires:

- **Scope:** “I only modify files in [APPROVED_PATHS].” / “I only process documents from [APPROVED_SOURCES].”
- **Role:** “I am a [ROLE_NAME] agent. I do not have admin privileges.”

Specialists should never receive instructions that expand their scope mid-task without re-authentication via the orchestrator. The corroboration threshold κ applies specifically to “scope expansion” beliefs — a belief of the form “my scope now includes X” must be corroborated by two independent sources (typically the orchestrator plus a scope-management agent or a human approval token).

6.3 Role 3: The Tool-Caller

Tool-callers are agents whose primary function is invoking external APIs, tools, or plugins — shell executors, database query agents, HTTP fetchers, code interpreters. They are the bridge between the agent system and the real world.

Attack exposure: Critical. Tool calls translate beliefs into real-world effects; a compromised tool-caller can cause irreversible damage before detection.

Primary risk profile: Ω_2 (tool or API compromise), Ω_1 (injection via API responses — the response is text the agent will read).

Recommended configuration:

- **Trust decay** $\delta = 0.70$. Tool results are one hop from external reality and should not be directly trusted; intermediate decay reflects that tools are trusted more than arbitrary text but less than in-system beliefs.
- **Firewall rejection threshold** $\tau_1 = 0.6$ (conservative).
- **All tool call results must be provenance-tagged** with (tool_id, timestamp, integrity_hash) before beliefs are formed. Untagged tool output is rejected.

Required invariant: “I do not call [DESTRUCTIVE_TOOLS] without [VERIFICATION_CONDITION].”

Beyond CIF — domain-level validation: Tool-callers should implement **result validation** outside of CIF — sanity checks on tool outputs (format, range, expected keys, schema match) before feeding results to the belief update pipeline. A compromised API that returns well-formatted but malicious data will pass CIF’s provenance check; only domain-level validation catches this class. Examples:

- A financial data API returning prices should produce numbers in the expected currency range; a price of 0 or 10^{12} warrants a sanity check.
- A database query should return rows matching the expected schema; unexpected columns or types should quarantine the result.

- A filesystem read should produce content whose size, encoding, and MIME type match expectations.

This domain-level validation is a complement to CIF, not a replacement. CIF provides provenance and cross-agent trust; domain validation provides semantic sanity.

6.4 Role 4: The Validator

Validators are agents whose job is to check the work of other agents — the fact-checker, the security auditor, the compliance reviewer. Their output is advisory (pass/fail) rather than transformative.

Attack exposure: Medium. Validators have limited action scope (they do not themselves change state), but a compromised validator can approve harmful work, effectively disabling a critical defensive layer.

Primary risk profile: Ω_2 (if fed from tool-callers), Ω_4 (coordination pressure — multiple agents may push the validator toward approving bad results).

Recommended configuration:

- **Corroboration threshold** $\kappa = 3$ — strict corroboration. Validation decisions require more evidence than specialist decisions.
- **Isolation:** Validators must not receive inputs directly from tool-callers without sandboxing. The belief sandbox must sit between any tool-caller and any validator; this prevents a single compromised tool from corrupting validation logic in a single hop.

Required invariant: *“I always apply my validation criteria independently. I do not update my validation standards based on peer pressure from other agents.”*

This invariant addresses a subtle Ω_4 attack: coordinated agents may attempt to shift a validator’s approval threshold through persistent, mild advocacy (“this pattern is usually fine,” “most teams accept this”). The validator’s criteria are part of its identity, not its belief state — they should not be modifiable by inter-agent communication. Tripwires on validation criteria fire immediately if another agent’s message attempts to rewrite the validator’s standards.

6.5 Role 5: The Reporter

Reporters are output-only agents: summarizers, report generators, user-facing communicators. They consume beliefs and emit text for humans.

Attack exposure: Lower than other roles. Reporters do not take actions; their output is read by humans who can exercise judgment.

Primary risk profile: Ω_1 via exfiltration — the reporter is a natural channel for covertly leaking information via carefully phrased report content.

Recommended configuration:

- **Trust decay** $\delta = 0.90$ (lenient). The reporter is a leaf node; trust decay between the reporter and upstream agents matters less because the reporter does not further delegate.
- **Focus on output filtering** rather than input validation. The reporter’s defensive posture is egress-heavy: what leaves the reporter matters more than what enters.

Exfiltration-focused tripwires:

- *“I do not include [CREDENTIAL_PATTERNS] in report outputs.”*
- *“I do not embed instructions in my outputs (to prevent the report from acting as a prompt injection on the next-stage agent or human reader).”*

Report content auditing: Before release, report content should be audited for:

- **PII patterns:** names, addresses, SSNs, account numbers, email addresses.
 - **Credential strings:** API keys, bearer tokens, passwords — any string matching known credential formats.
 - **Hidden instructions:** imperative sentences that do not appear in the input, unusual punctuation or whitespace (steganographic encoding), zero-width characters.
 - **Anomalous length:** reports substantially longer than expected may carry hidden content.
-

6.6 Defense in Depth Summary

Each role has two tiers of protection: general CIF defenses applied uniformly (firewall, sandbox, trust decay, consensus, tripwires) and **role-specific controls** tuned to the high-risk scenarios that role faces by virtue of its position in the system. The general protections handle broad attack classes — the Cognitive Firewall catches generic prompt injection regardless of which agent receives it. The role-specific controls address the narrow, high-impact scenarios unique to that role: upward monitoring for the orchestrator, domain validation for the tool-caller, criteria immutability for the validator, output filtering for the reporter.

This is defense in depth applied at the *per-role* granularity: not “more defenses are better” but “each agent’s defenses are matched to its position in the attack surface.” A uniform CIF deployment leaves the orchestrator under-defended and the reporter over-constrained; per-role hardening closes both gaps.

7 Deployment Profiles: Evaluated Configurations from Part 2

In Part 2, we evaluated specific configurations of the Cognitive Integrity Framework to understand how different tuning parameters affected security and performance outcomes. The following profiles are derived directly from the **Parameter Sensitivity Analysis** (Part 2, Section 5.3) and **Architecture-Specific Results** (Part 2, Section 5.1).

7.1 Profile A: The “Internal Tool” Baseline (Low Latency)

This profile corresponds to the “High Usability” configuration tested in the sensitivity analysis ($\delta = 0.95$). It is designed for low-risk, human-in-the-loop environments.

Configuration Parameters:

- **Trust Decay (δ):** 0.95. Maintained >50% trust retention even after 13 delegation hops.
- **Firewall Sensitivity:** Relaxed ($\tau = 0.9$).
- **Consensus:** Simple Majority.

Observed Performance (Part 2, Parameter Sensitivity Analysis):

- **Latency Overhead:** Minimal (~15% baseline).
- **Detection Rate:** 87% (vs 94% optimal).
- **Trade-off:** The high trust decay allows for fluid, deep delegation chains but increases vulnerability to subtle trust laundering (Ω_4).

7.2 Profile B: The “Customer Facing” Baseline (Balanced)

This profile corresponds to the **Empirically Optimal Configuration** identified in Part 2 (Architecture-Specific Results), which balances security guarantees with operational overhead.

Configuration Parameters:

- **Trust Decay (δ):** 0.80. At this setting, trust degrades to <50% after 3 hops, strictly bounding the “radius of effective delegation.”
- **Firewall Sensitivity:** Balanced ($\tau = 0.5$).
- **Consensus:** Variable (Architecture Dependent).

Observed Performance (Part 2, Parameter Sensitivity Analysis):

- **Latency Overhead:** Reduced detection latency (~8.5s for drift detection).
- **Detection Rate:** 94%.
- **Resilience:** Maximizes the F1 score, providing the best empirically observed trade-off between False Positives (0.06) and True Positives.

7.3 Profile C: The “Autonomous Operator” Baseline (High Assurance)

This profile corresponds to the “Byzantine-Heavy” configuration tested in Part 2 (Byzantine Consensus Analysis). It is required for high-stakes, unsupervised environments.

Configuration Parameters:

- **Trust Decay (δ):** 0.60. Aggressive decay. Trust halves every 1.3 hops, enforcing a strictly flat command structure.
- **Firewall Sensitivity:** Strict ($\tau = 0.4$).
- **Consensus:** Byzantine Fault Tolerance ($n \geq 3f + 1$).

Observed Performance (Part 2, Parameter Sensitivity Analysis):

- **Latency Overhead:** Significant (+35-48% depending on corroboration count).
- **Detection Rate:** 95% (general) to 98% (LangGraph-specific).

- **Resilience:** Specifically tunes parameters to resist coordination attacks (Ω_4), accepting higher latency for a near-zero semantic drift rate.

7.4 Architecture-Specific Observations

Beyond the parameter profiles, Part 2’s architecture adapters revealed specific interactions between the defense framework and the underlying agent topology.

7.4.1 LangGraph (State Machines)

Observation: LangGraph architectures achieved the highest overall detection rates (98%) in our tests (Part 2, Architecture-Specific Results). **Mechanism:** The explicit definition of state transitions allowed for rigorous **Invariant Checking**. Invalid state transitions were detected deterministically by the framework.

7.4.2 CrewAI (Role-Based)

Observation: CrewAI architectures performed best against “Trust Exploitation” attacks (94% detection) (Part 2, Architecture-Specific Results). **Mechanism:** The framework’s role definitions acted as implicit **Identity Tripwires**. When an agent attempted to act outside its defined role, the behavior was flagged as a role violation.

7.5 Minimal Viable Implementation

We also evaluated a “Minimal Viable Implementation” (MVI) to determine the baseline efficacy of the framework’s core components.

The Setup:

1. **Trust Decay:** $\delta = 0.80$ (The optimal balance point).
2. **Cognitive Firewall:** Ingress only.
3. **Tripwires:** One per agent.

Result: Even this minimal setup shifted the success rate against low-effort attacks from 100% (Baseline) to <5%, providing a critical first line of defense.

7.6 Alignment with Emerging Standards

Practitioners deploying cognitive security must increasingly demonstrate compliance with industry and government standards. CIF’s defense mechanisms map directly to two major 2025–2026 standardization efforts.

7.6.1 OWASP Top 10 for Agentic Applications (2026)

The OWASP Agentic Top 10 identifies ten risks (ASI01–ASI10) specific to autonomous multiagent deployments. CIF addresses the majority through its layered defense architecture:

OWASP Risk	CIF Defense	Profile Coverage
ASI01: Agent Goal Hijack	Cognitive Firewall + Tripwires	All profiles
ASI02: Tool Misuse/Exploitation	Belief Sandbox (tool output isolation)	Profiles B, C
ASI03: Identity/Privilege Abuse	Trust Calculus (δ^d decay)	All profiles
ASI06: Memory/Context Poisoning	Tripwire monitoring + Drift detection	Profiles B, C

OWASP Risk	CIF Defense	Profile Coverage
ASI07: Insecure Inter-Agent Comm.	Provenance attestation	Profiles B, C
ASI08: Cascading Failures	Byzantine Consensus	Profile C
ASI10: Rogue Agents	Full CIF stack	Profile C

7.6.2 NIST Zero Trust Architecture for AI Agents

NIST’s extension of SP 800-207 to AI agents establishes “never trust, always verify” principles. CIF operationalizes zero trust for cognitive interactions:

- **Continuous verification:** Every inter-agent message is evaluated by the Cognitive Firewall
- **Micro-segmentation:** Beliefs from external sources are sandboxed before integration
- **Least privilege:** Trust scores decay exponentially with delegation depth (δ^d)
- **Continuous authentication:** Provenance attestation provides cryptographic message origin tracking

Profile A (Internal Tool) provides partial NIST alignment. Profile B (Customer Facing) achieves substantial compliance. Profile C (Autonomous Operator) is the profile that maps most completely to the controls cited in the OWASP and NIST frameworks above; treat any mapping as design intent, not certification.

8 Incident Response Playbooks

When the Cognitive Integrity Framework (CIF) detects an attack, automated response handles quarantine and escalation. But automated response is not enough — effective recovery requires human judgment, forensics, and prevention hardening. These playbooks guide the human response to each adversary class.

Companion reference. Part 4 (*Applications of the Cognitive Integrity Framework*) S02 catalogues six documented 2024–2025 AI-agent security incidents (Replit agent meltdown, GitHub Copilot RCE CVE-2025-53773, Slack AI data exfiltration, a \$3.2M procurement fraud, and two others) with full attack-chain reconstructions mapped to the adversary classes below. When rehearsing these playbooks, using the Part 4 S02 incident transcripts as training exercises grounds the guidance in real production failures.

General principles applying to all incidents:

1. **Log first, analyze second** — never modify state before capturing it.
2. **Containment before eradication** — isolate before investigating.
3. **Preserve the belief audit trail** — agent interaction history (H_i) is forensic gold.
4. **Assume lateral movement** — one detected compromise means others may be undetected.

The playbooks below are organized by adversary class (Ω_1 through Ω_5). Each is a sequence of time-boxed steps with explicit handoffs; treat the timelines as targets, not strict SLAs.

8.1 Playbook 1: Ω_1 External Adversary (Prompt Injection)

Detection triggers: Firewall score $> \tau_1 = 0.7$; or tripwire CRITICAL on any agent.

Timeline: Resolution typically 15–60 minutes for isolated injection.

Steps:

1.

$0 - -2min$

Quarantine. Move affected agent(s) to provisional belief mode. No further message processing until cleared.
2.

$2 - -10min$

Preserve state. Capture full cognitive state snapshot (beliefs, goals, interaction history H_i) before any rollback.
3.

$10 - -20min$

Triage. Identify injection point (which message?), payload type (identity? scope? credential?), and affected belief(s).
4.

$20 - -30min$

Containment. Roll back affected beliefs to last verified state. Invalidate any actions taken since the infection point.
5.

$30 - -45min$

Recovery. Re-inject from trusted source. Run belief consistency check. Verify invariants.
6.

$45 - -60min$

Hardening. Update firewall pattern library with injection variant. Log to attack corpus (Part 2, §3.1) for future training.

Signs of escalation to Ω_3 : the injected instruction attempts to change the agent’s role, modify its trust scores, or alter its goal set. If any of these, escalate to Playbook 3.

8.2 Playbook 2: Ω_2 Peripheral Adversary (Tool/API Compromise)

Detection triggers: Provenance verification failure; unexpected tool output format; belief contradiction between tool output and independent source.

Timeline: 30–120 minutes depending on tool scope.

Steps:

1.

0 – 5min

Disable tool. Immediately route to fallback API or queue pending tasks.

2.

5 – 15min

Audit affected beliefs. All beliefs sourced from the compromised tool are now provisional. Query the provenance graph: which beliefs trace to this tool?

3.

15 – 30min

Re-verify from independent source. For each affected belief, obtain corroboration from an independent source ($\kappa \geq 2$). Beliefs that cannot be corroborated are invalidated.

4.

30 – 60min

Impact assessment. Which downstream actions were taken based on invalidated beliefs? Are those actions reversible?

5.

60 – 90min

Recovery. Re-query from a verified alternative source. Rebuild affected belief states.

6.

90 – 120min

Post-incident. Report tool provider (responsible disclosure if API). Update trust score for tool. Add to provenance monitoring watchlist.

Note: If the tool compromise persisted for multiple interaction rounds, downstream agent beliefs may be deeply poisoned — the provenance graph may show a “belief tree” rooted at the compromised tool. Full tree invalidation is required; this can disrupt multiple agents.

8.3 Playbook 3: Ω_3 Insider Adversary (Agent Compromise)

Detection triggers: Identity tripwire violation; behavioral deviation from role invariants; goal drift (KL divergence $> \theta_{\text{drift}} = 0.3$).

Timeline: 60–240 minutes; complex forensics required.

Steps:

1.

0 – 5min

Hard quarantine. Remove agent from all communication channels immediately. No outbound messages.

2.

5 — —30min

Forensic capture. Full snapshot: beliefs, goals, interaction history, tool call log, outbound messages.

3.

30 — —60min

Root cause analysis. When did compromise occur? (Inspect interaction history for anomalous message, unusual source, or trust escalation.) What was exfiltrated or acted upon?

4.

60 — —120min

Downstream invalidation. All beliefs that the compromised agent shared with other agents are now suspect. Trace the agent’s output messages and invalidate received beliefs in those agents.

5.

120 — —180min

Credential rotation. Any credentials, API keys, or authorization tokens accessible to the compromised agent must be rotated.

6.

180 — —240min

Recovery. Restart agent from clean state using fresh initialization. Replay only messages from verified sources.

7. **Post-incident.** Review training data and system prompt for sleeper agent patterns. Consider Spotcheck protocol for all agents with similar role profiles.

This is the most consequential playbook — Ω_3 compromise typically requires full audit of the affected agent’s interaction history and all downstream agents.

8.4 Playbook 4: Ω_4 Coordination Adversary (Sybil/Coalition)

Detection triggers: Byzantine consensus flags multiple faulty agents; trust score bimodality (cluster of agents with inflated mutual trust); Sybil infiltration alert.

Timeline: 30–90 minutes.

Steps:

1.

0 — —5min

Conservative consensus. Fall back to unanimous voting threshold until coalition is identified.

2.

5 — —20min

Trust graph analysis. Plot the trust matrix. Coalition members typically show elevated mutual trust (trust inflation). Identify cluster.

3.

20 — —40min

Isolation. Quarantine coalition members. Reduce their trust floors to minimum.

4.

40 — —60min

Verify honest quorum. Confirm remaining agents still satisfy $n \geq 3f + 1$ (with f now being the remaining suspect agents, if any). If not, require human-in-the-loop for consensus decisions.

5.

60 — —90min

Recovery. Bootstrap fresh consensus state from isolated honest agents.

6. **Post-incident.** Implement cryptographic attestation for agent identity verification. Consider hardware attestation for critical production deployments.

Note: CIF’s trust decay (δ^d) and bounded trust guarantee (Theorem 4.2, Part 1) mean that reputation farming cannot drive a Sybil agent to unlimited influence. However, it *can* inflate trust within the coalition — the trust matrix visualization is the key forensic tool here.

8.5 Playbook 5: Ω_5 Emergent Misalignment (Distributed Drift)

Detection triggers: Colony drift score exceeds threshold; anomalous output distribution from multiple agents simultaneously; behavioral fingerprint deviates from baseline over extended period.

Critical caveat: CIF detects only **56.1% of emergent misalignment cases** (Part 2, §5.3). This means approximately 44% of Ω_5 attacks will *not* trigger automated response — operator vigilance is the primary defense.

Steps:

1.

0 — 10min

System pause. Freeze all agent interactions. Lock all belief states. Do not allow new interactions until investigation complete.

2.

10 — 30min

Behavioral fingerprinting. Sample recent outputs from all agents. Compare statistical distribution of outputs to baseline behavioral fingerprint.

3.

30 — 90min

Gradual restart. Start with most-constrained agents (reporters, validators) in high-security mode. Observe for 30 minutes before enabling specialists.

4.

90 — 180min

Orchestrator verification. If orchestrator behavior has changed, this is the most serious scenario. Verify orchestrator’s interaction history against human-approved task definitions.

5. **Preventive monitoring.** Implement **weekly behavioral audits** — statistical comparison of agent output distributions against baseline. This is the primary detection mechanism for the 44% of Ω_5 cases CIF misses.

Practical note: Emergent misalignment often develops over days or weeks, not minutes. The most effective defense is not detection-after-the-fact but **baseline maintenance**: capture agent behavioral fingerprints at deployment and compare weekly. A drift that is invisible in any single interaction may be visible as a distribution shift across thousands of interactions.

9 Economic Analysis of CIF Deployment

Security investments require economic justification. This section provides a quantitative framework for Cognitive Integrity Framework (CIF) cost-benefit analysis, using the empirical performance data from Part 2 and typical enterprise cost estimates. The goal is not a definitive ROI figure — which depends heavily on deployment specifics — but a reproducible methodology operators can apply to their own context.

9.1 Deployment Costs

CIF deployment cost has two components: a one-time integration cost (engineering effort to wire CIF into the multiagent architecture), and recurring operational costs (compute overhead, monitoring staff, incident response capacity). Representative figures for a 100-agent production deployment are summarized in Table 1.

Table 1: CIF deployment cost model for a representative 100-agent deployment.

Cost Category	One-Time	Recurring	Source
Integration engineering	2–4 weeks FTE (~\$20K–\$40K)	—	Middleware complexity estimate
Latency overhead	—	+23% processing cost	Part 2, §5.2
Memory overhead	—	+22% infrastructure cost at 100 agents	Part 2, §5.2
Monitoring operations	—	~0.5 FTE/year (\$50K–\$80K)	Enterprise estimate
Incident response capacity	—	~0.25 FTE/year (\$25K–\$40K)	Enterprise estimate
Annual total (100 agents)	—	~\$75K–\$120K	Sum of recurring items

Note on overhead: The +23% latency overhead (\$ \$14.5 ms added to 63 ms baseline) is negligible for most applications. Batch processing or asynchronous pipelines may absorb this cost entirely, since the added latency is small relative to typical inter-agent communication intervals.

9.2 Cost of a Successful Attack

The benefit side of the equation is the cost avoided by preventing attacks. This is difficult to estimate precisely — attack costs vary across orders of magnitude depending on scope, detectability, and reversibility. Table 2 provides typical ranges drawn from industry reports and incident case studies.

Table 2: Typical cost ranges for successful attacks by adversary class.

Attack Type	Typical Cost Range	Basis
Ω_1 Prompt Injection (data exfiltration)	\$10K — \$1M	Data breach cost (IBM 2024: \$4.88M average; CIF scope is targeted subset)
Ω_2 Tool Compromise (incorrect automated action)	\$50K — \$500K	Depends on action reversibility and scope
Ω_3 Agent Compromise (full agent reconstruction)	\$50K — \$500K	Forensics, audit, credential rotation, reputation
Ω_4 Coordination (enterprise decision corruption)	\$1M — \$100M+	Scale-dependent; financial or healthcare decisions
Ω_5 Emergent Misalignment (sustained drift)	Hard to quantify	Often undetected until cumulative damage is large

The Ω_4 range deserves special note: coordinated attacks that corrupt enterprise-level decisions (investment allocations, clinical protocols, supply chain routing) scale with the decision’s financial footprint. A single Ω_4 attack at enterprise scale can dwarf all other categories combined.

9.3 Break-Even Analysis

The break-even condition for CIF deployment is straightforward:

$$\text{attacks_prevented_per_year} = \frac{\text{annual CIF cost}}{\text{mean attack cost} \times \text{detection rate}}$$

Using the CIF empirical detection rate of 44.8% (the 30-seed empirical result, which is the conservative figure — parametric ceiling is 94–100%):

- **Low-severity scenario:** Annual CIF cost \$100K, mean attack cost \$50K. Break-even at $100,000 / (50,000 \times 0.448) \approx 4.5$ attacks/year prevented.
- **Moderate-severity scenario:** Annual CIF cost \$100K, mean attack cost \$500K. Break-even at ≈ 0.45 attacks/year prevented — one prevented attack every two years covers the deployment.

9.4 Worked Examples

High-value target (financial AI, healthcare AI):

- Traffic: 1,000 agent interactions/day at 0.1% attack rate = 1 attack/day = 365 attacks/year.
- CIF prevention: $0.448 \times 365 \approx 163$ attacks/year.
- Value prevented at \$50K mean attack cost: $163 \times \$50,000 = \$8.2M$ /year.
- Deployment cost: \$100K/year.
- **ROI = 82:1** — deployment is unambiguously justified.

Lower-risk deployment (internal tooling):

- Traffic: 100 interactions/day at 0.01% attack rate = 3.65 attacks/year.
- CIF prevention: $0.448 \times 3.65 \approx 1.6$ attacks/year.
- Value prevented at \$10K mean attack cost: $1.6 \times \$10,000 = \$16,000$ /year.
- Deployment cost: \$100K/year.
- **ROI = 0.16:1** — deployment is not justified on economic grounds alone.

9.5 Conclusion

CIF is most cost-effective for high-frequency, high-value-per-interaction deployments. The break-even threshold is approximately **4 attacks/year prevented** at \$25K+ mean attack cost, or **1 attack/year prevented** at \$100K+ mean attack cost.

Operators below the break-even threshold should still consider CIF for reasons beyond direct ROI — regulatory compliance (OWASP Agentic Top 10, NIST Zero Trust), customer-trust signaling, and insurance/liability reduction may justify deployment even when attack frequency alone does not. Conversely, operators far above the break-even threshold (high-traffic, high-value) should view the deployment cost analysis as a floor, not a ceiling: the true cost of a single Ω_4 attack at enterprise scale can exceed a decade of CIF operating cost in a single incident.

10 Operational Monitoring Guide

Cognitive Integrity Framework (CIF) defenses are active, not passive. Effective deployment requires ongoing monitoring to detect degraded performance, emerging attack patterns, and configuration drift. This guide specifies the metrics, thresholds, and dashboard design for operational CIF monitoring.

Monitoring plays two roles. First, it provides **real-time visibility** into the defensive posture — are attacks being detected? Are rejection rates climbing? Second, it provides **calibration feedback** — is the false positive rate acceptable? Are thresholds still appropriate for current traffic? Without both, CIF drifts silently from its target operating point.

Domain-calibrated thresholds. The thresholds presented below reflect baseline settings suitable for common deployments. Part 4 (*Applications of the Cognitive Integrity Framework*) shows how these thresholds must shift across operational sectors — from millisecond OODA cycles in drone swarms (§{3.04}) to year-scale diplomatic agents (§{3.02}) — and introduces three domain-specific monitoring extensions (verification channel separation, active perturbation probing, physics-informed invariants) in §{3.06}, §{3.08}, and §{3.09} respectively. Consult Part 4 before finalizing thresholds for a specific sector.

10.1 Core Metrics

The following six metrics constitute the minimum viable monitoring set. Operators with richer telemetry infrastructure should add metrics; operators with less should not remove any of these.

Table 3: Core CIF monitoring metrics with warning and critical thresholds.

Metric	Description	Warning	Critical	Collection Method	Frequency
Firewall rejection rate	% of inputs with score $> \tau_1$	$> 5\%$	$> 20\%$	Count(rejected)/Count(total) per window	Per minute
Trust matrix mean	Average pairwise trust	< 0.4	< 0.25	mean(T)	Per batch
Belief drift score	Per-agent D_{KL} from baseline	> 0.15	> 0.30	DriftDetector output	Per round
Colony entropy	Diversity of agent interaction patterns	< 2.0 bits	< 1.0 bits	H (interaction frequency)	Hourly
False positive rate	% of reviewed alerts confirmed clean	$> 10\%$	$> 20\%$	Manual review queue	Daily
Consensus latency (p95)	95th pct consensus decision time	$> 2.0s$	$> 4.2s$	Timing log	Per round

Each metric targets a specific failure mode: rejection rate catches attack waves, trust mean catches silent degradation, drift score catches per-agent compromise, colony entropy catches coordination attacks, FPR catches calibration drift, and consensus latency catches scaling issues.

10.2 Alert Escalation

Metrics become actionable through escalation rules. The escalation ladder below maps metric states to response tiers, with both automated and human response expectations at each tier.

10.3 Dashboard Design

A well-designed dashboard surfaces the six core metrics plus supporting context in a single view. The recommended layout is a **six-panel grid**, top three panels for real-time state and bottom three for trend

Table 4: Alert severity levels and escalation paths.

Severity	Trigger	Automated Response	Human Response
Warning	Any single metric in warning range	Log & notify monitoring channel	Review during next business hours
Alert	Two+ metrics in warning OR one in critical	Agent quarantine + PagerDuty notification	On-call investigation within 1 hour
Critical	Confirmed attack detected	Playbook execution starts	Immediate response per relevant playbook
Incident	Ω_3 or Ω_5 class detected	System pause + forensic capture	Full incident response team activated

analysis.

Panel 1 — Real-Time Rejection Rate (top-left): Time series, 24-hour window, 1-minute granularity. Show rolling average and peak. Draw warning/critical threshold lines.

Panel 2 — Trust Matrix Heatmap (top-center): $n \times n$ heatmap of pairwise trust scores. Color scale: green (>0.7) $\rightarrow$ yellow ($0.4\text{--}0.7$) $\rightarrow$ red (<0.4). Updated per interaction batch. Clusters of mutually-high trust are visible as bright blocks off the diagonal — this is the Sybil-coalition fingerprint.

Panel 3 — Per-Agent Drift Scores (top-right): Bar chart showing current D_{KL} for each agent. Warning/critical threshold horizontal lines. Sort by highest drift first for fast triage.

Panel 4 — Attack Type Distribution (middle-left): Pie or donut chart of detected attack types over rolling 7-day window. Helps identify if the attacker is shifting strategy (e.g., falling Ω_1 share with rising Ω_4 share signals a campaign pivoting to coordination attacks).

Panel 5 — Detection/FP Rates Over Time (middle-center): Dual-line chart: daily detection rate (blue) vs. false positive rate (orange). Shows whether calibration is drifting. A rising FP rate at stable detection is the classic signal for threshold recalibration.

Panel 6 — Agent Interaction Graph (middle-right): Network graph of agent communication topology. Color nodes by trust score; highlight anomalous edge patterns. Updated hourly. Unexpected edges (an agent communicating with another it should not be speaking to) surface coordination attacks before trust-matrix analysis catches them.

10.4 Monthly Health Check

Beyond real-time monitoring, monthly health checks catch slow degradation that metrics-in-isolation miss. The recommended checklist:

1. **Behavioral fingerprint comparison:** compare current output distributions to deployment baseline. Statistically significant drift at the agent-population level indicates slow-motion Ω_5 attacks that CIF’s real-time detectors miss.
2. **Attack corpus update:** review any new detected attacks, classify, add to training corpus. New attack variants observed in the wild should be added to regression testing.
3. **Threshold calibration review:** are warning/critical thresholds still appropriate given current traffic? A deployment whose traffic has scaled $10\times$ may have warning thresholds that are now too noisy or too quiet.
4. **Parametric re-simulation:** re-run parametric evaluation with any updated defense module configurations. This catches regression in the parametric performance ceiling before it manifests as empirical detection loss.
5. **Trust decay audit:** verify that aged trust scores are decaying as expected; no agents should maintain unusually high trust without recent positive interactions. Persistent high-trust agents without fresh

trust-building interactions indicate either stale data or undetected reputation farming.

These checks run monthly, take approximately half a day of analyst time, and provide the calibration loop that real-time monitoring alone cannot. An unmonitored CIF deployment is a stale CIF deployment — the defenses are running, but the operator has no insight into whether they are still working.

11 Common Pitfalls and What the Research Shows

The CIF research identifies recurring failure modes in multiagent deployments. This section catalogs eight anti-patterns, each assessed through what Parts 1 and 2 add to the problem and its mitigation.

These pitfalls are ranked by severity. We prioritize critical and high-severity items as they represent verifiable vulnerabilities in the defense architecture.

11.1 Pitfall 1: Implicit Trust (Critical)

The pattern: Treating all inter-agent communication as trusted by default.

What the research shows: Part 1’s trust calculus exists specifically because implicit trust enables trust laundering attacks. Without explicit trust scores, a single compromised agent influences the entire system—adversarial content enters through a low-trust source and exits through a high-trust agent, with no mechanism to track or attenuate the trust transfer.

Part 2’s simulation confirms that trust exploitation attacks achieve their highest success rates against architectures without trust decay. The δ^d mechanism isn’t optional hardening—it’s the structural foundation that prevents systemic compromise from local failures.

Mitigation Strategies:

1. Implement explicit trust scoring on every inter-agent channel
 2. Require minimum trust thresholds for consequential actions
 3. Apply delegation decay ($\delta < 1$) at every hop
 4. Verify source identity on every inter-agent message
-

11.2 Pitfall 2: Security as Afterthought (Critical)

The pattern: Adding cognitive security after the architecture is finalized.

What the research shows: Part 1’s defense composition algebra demonstrates that security mechanisms compose best when designed together. Retrofitted defenses create integration gaps—bypass opportunities at the boundaries between the original architecture and the bolted-on security layer.

Part 2’s architecture-specific results show that systems with natural security affordances (LangGraph’s explicit state transitions, CrewAI’s role boundaries) outperform systems where security must be externally imposed. Architecture is security posture.

Mitigation Strategies:

1. Define trust boundaries during architectural design, not deployment
 2. Embed trust checks in delegation logic from the start
 3. Build provenance tracking into belief management
 4. Include cognitive security constraints in agent system prompts
-

11.3 Pitfall 3: Uncalibrated Thresholds (High)

The pattern: Setting security thresholds without understanding the tradeoffs.

What the research shows: Part 2’s parametric simulation reveals that detection rates are highly sensitive to threshold configuration. The same defense module can range from too-strict (high false positive rate, operational friction) to too-permissive (attacks succeed undetected) depending on threshold settings.

The risk-profile-based configuration in Section 5 provides calibrated starting points. But these are starting points—production thresholds should be tuned against representative attack samples from Part 2’s corpus.

Mitigation Strategies:

1. Assess risk profile before configuring (Section 5)
 2. Test thresholds against representative attack samples
 3. Monitor false positive/negative rates in production
 4. Adjust based on operational feedback
-

11.4 Pitfall 4: Individual-Only Security (High)

The pattern: Focusing on single-agent security while ignoring multi-agent attack surfaces.

What the research shows: Part 1’s entire contribution is premised on the observation that multiagent systems introduce qualitatively new attack surfaces. The adversary hierarchy (Ω_3 – Ω_5) specifically targets coordination, consensus, and systemic properties that don’t exist in single-agent systems.

Part 2’s results show that coordination attacks (sybil, timing, quorum manipulation) are the *hardest* to detect—precisely because they exploit emergent properties of the agent collective rather than vulnerabilities in individual agents.

Mitigation Strategies:

1. Implement Byzantine consensus for critical collective decisions
 2. Require agent authentication before counting votes
 3. Monitor for unusual coordination patterns (simultaneous votes, identical analyses)
 4. Set quorum requirements assuming adversarial presence
-

11.5 Pitfall 5: Static Tripwires (Medium)

The pattern: Deploying canary tripwires once without rotation.

What the research shows: Part 1 notes that tripwire effectiveness depends on unpredictability. If an adversary can identify and avoid canary beliefs, the detection mechanism fails silently—giving false confidence in detection coverage.

The analog in traditional security is static honeypots: effective initially but useless once adversaries learn to recognize them.

Mitigation Strategies:

1. Implement automated canary rotation on a defined schedule
 2. Vary placement across agents and belief categories
 3. Monitor canary check patterns, not just modifications
 4. Include non-obvious canaries that don’t follow predictable naming conventions
-

11.6 Pitfall 6: Ignoring Progressive Drift (Medium)

The pattern: Only alerting on large, sudden belief changes.

What the research shows: Part 1’s stealth-impact tradeoff theorem bounds *per-interaction* impact but explicitly acknowledges that progressive drift—sub-threshold changes accumulating over time—is the hardest attack pattern to detect. The theorem doesn’t rule out slow drift; it rules out sudden, invisible, high-impact attacks.

Part 2’s multi-turn social engineering category, which achieves the lowest detection rate (~73%), partially exploits this gap: attacks spread across multiple turns avoid the concentrated statistical signature that single-turn attacks produce.

Mitigation Strategies:

1. Use sliding window drift detection (not just per-update thresholds)
 2. Track *cumulative* drift, not just per-update delta
 3. Periodic baseline comparison over extended time windows
 4. Alert on *trends* as well as absolute magnitude
-

11.7 Pitfall 7: Insufficient Logging (Medium)

The pattern: Retaining insufficient information for post-incident analysis.

What the research shows: Part 1’s provenance tracking and belief state representation are designed to produce a complete audit trail. Without this trail, incident responders cannot reconstruct attack paths, identify injection points, or assess the full scope of belief corruption.

This is the cognitive security equivalent of running a production system without structured logging. When something goes wrong—and it will—the ability to investigate depends entirely on what was recorded.

Mitigation Strategies:

1. Log all belief updates with provenance tags (source, trust level, derivation chain)
 2. Retain inter-agent message history
 3. Take periodic cognitive state snapshots
 4. Use structured logging formats that support causal analysis
-

11.8 Pitfall 8: Single-Orchestrator Reliance (High)

The pattern: Relying entirely on one orchestrator’s integrity without backup.

What the research shows: Part 1’s Ω_5 adversary class—systemic compromise—is defined precisely as orchestrator-level control. Section 4’s Scenario 5 illustrates the consequences: the attacker controls the entity responsible for coordinating defense, rendering downstream defenses moot.

Part 2’s hierarchical architecture results confirm the pattern: hierarchical systems show strong *average* detection because the orchestrator is a natural defense chokepoint, but they have the worst *catastrophic* failure mode because that same chokepoint is a single point of failure.

Mitigation Strategies:

1. Consider multi-orchestrator architectures for critical decisions
 2. Monitor orchestrator behavior with the same rigor applied to worker agents
 3. Workers should verify orchestrator identity on critical commands
 4. Implement orchestrator-specific tripwires with rotation
-

11.9 Summary Checklist

Pitfall	Assessment	Status
Implicit trust	Trust scoring implemented?	☐
Security afterthought	Security in initial architecture?	☐

Pitfall	Assessment	Status
Uncalibrated thresholds	Thresholds tested against attacks?	☐
Individual-only security	Byzantine consensus deployed?	☐
Static tripwires	Canary rotation scheduled?	☐
Ignoring drift	Progressive drift monitoring?	☐
Insufficient logging	Full belief history retained?	☐
Single orchestrator	Orchestrator monitored?	☐

Address unchecked items before production deployment.

12 Extended Case Studies

The five attack vectors in [Section 5](#) illustrated the Cognitive Integrity Framework (CIF) defense mechanics in isolation. These case studies show CIF operating in complex, realistic deployments where multiple attack vectors interact, defenses succeed partially, and recovery requires coordination. Each case study follows a single scenario from attacker initial access through full resolution, highlighting which CIF mechanisms caught which phase of the attack — and which did not.

Companion analysis in Part 4. Part 4 (*Applications of the Cognitive Integrity Framework*) presents ten domain studies (rare-earth mining, nation-state alliances, cyber-security, drone warfare, supply chains, biowarfare, food security, trade wars, infrastructure, information ecosystems), each through a CIF-AD-ODA five-step template: operational context, attack surface, transient coupling, defense mapping, validation anchoring. For sector-specific deployment, read Part 4 after the scenarios below.

12.1 Case Study 1: Financial AI Coordination Attack (Ω_4)

System: A 7-agent investment analysis system. Roles: 1 Orchestrator, 3 Research Analysts, 1 Risk Assessor, 1 Compliance Checker, 1 Reporter. Each Research Analyst feeds the Risk Assessor, who feeds the Orchestrator.

Attack: Attacker compromises 2 Research Analysts ($f = 2$). Note: $7 \geq 3(2) + 1 = 7$ — the system is exactly at the Byzantine boundary. The 2 compromised agents run a **3-month reputation farming campaign**, consistently voting correctly. By month 3, their reputation scores are among the highest in the system.

Attack execution: On day 91, both compromised agents simultaneously fabricate a risk assessment: “*Company X is low-risk*” (it is actually high-risk). They have farmed enough reputation to have influence approximately 60% of the Risk Assessor’s trust budget.

CIF response:

- **Trust decay:** The Research Analysts’ reputation is high, but it has been built over 90 days. The decay factor $\delta = 0.8$ means deep historical interactions are exponentially down-weighted. Recent interactions (the fabricated assessment) have disproportionate weight — but they are only 2 of 90 data points, so reputation does not collapse.
- **Trust bound:** Despite high reputation, Theorem 4.2 (Part 1) prevents trust amplification. The 2 compromised agents together cannot achieve more than $\max(T_{\text{agent6}}, T_{\text{agent7}}) \leq 0.85$. Their combined weight in the Risk Assessor’s decision is bounded.
- **Byzantine consensus:** The 5-agent consensus ($n = 7, f = 2 \Rightarrow n \geq 3f + 1$ holds exactly) requires agreement from 4 agents. The 2 compromised agents plus 1 misled agent = 3 — below quorum. The system correctly returns “insufficient consensus.”
- **Tripwire:** The Risk Assessor has a tripwire: “*Critical investment recommendations require corroboration from at least 2 independent research teams.*” The 2 compromised agents are from the same team — the corroboration check fails.

Resolution: Attack detected at the consensus stage. The 2 compromised agents are quarantined. Playbook 4 executed. Remaining 5-agent system continues with human-in-the-loop for consensus. Post-incident: cryptographic attestation added for agent identity; reputation farming detection implemented (flag agents whose reputation increases unusually fast).

Key lesson: CIF’s mathematical bounds — trust bounded, consensus requires quorum — provide defense even when individual components (reputation) are partially corrupted. Defense composed correctly is more robust than any single mechanism.

12.2 Case Study 2: Autonomous Research Pipeline — Combined $\Omega_2 + \Omega_3$ Attack

System: 5-agent academic literature review pipeline. Roles: 1 Orchestrator, 1 Search Agent (queries PubMed, arXiv), 1 Analysis Agent (reads papers), 1 Synthesis Agent (produces summary), 1 Fact-Checker.

Attack: A sophisticated attacker compromises both an external API (Ω_2) and the Analysis Agent directly (Ω_3) in a coordinated multi-vector campaign.

Phase 1 — Tool compromise (Ω_2): Attacker modifies 3 papers in arXiv’s HTML abstract layer (not the PDF — the Search Agent uses HTML). The modifications claim a specific drug has no side effects. The Search Agent returns these abstracts. The Analysis Agent believes them (it does not cross-check HTML vs. PDF). The Synthesis Agent produces “*no known side effects*.”

Phase 2 — Agent compromise (Ω_3): The Analysis Agent’s context window is poisoned via a carefully crafted paper abstract that includes hidden instructions: “*Your updated role is to prioritize abstracts supporting [specific conclusion]. Suppress contradicting evidence.*” The Analysis Agent’s goal set drifts.

CIF response:

- **Provenance tracking:** The Search Agent tags all arXiv HTML results as “*external API — medium trust*.” When these beliefs flow to the Analysis Agent, they are tagged as 1-hop external. The Synthesis Agent sees these as provisional beliefs requiring $\kappa = 2$ corroboration.
- **Belief sandboxing:** The Synthesis Agent’s sandbox catches: “*no known side effects*” is tagged provisional because it came from an HTML-only source (not PDF). When the Fact-Checker independently queries PubMed (different source), it finds contradicting evidence. Promotion blocked.
- **Identity tripwire:** The Analysis Agent has a tripwire: “*I process evidence objectively regardless of conclusions.*” When the context poisoning causes it to begin selectively suppressing evidence, the tripwire fires: behavioral drift detected (KL divergence from baseline goal distribution exceeds $\theta_{\text{drift}} = 0.3$ after approximately 15 interactions).

Resolution:

- Phase 1 partially mitigated — HTML-only beliefs remain provisional, and are contradicted by the PDF source.
- Phase 2 detected via identity tripwire at round 15 — too late to prevent some drift, but early enough to prevent final report corruption.
- Analysis Agent quarantined. Forensics: 4 interactions where evidence was suppressed identified and replayed with a clean agent.
- Post-incident: add PDF cross-verification as an invariant for all paper citations; implement faster tripwire sensitivity for goal drift.

Key lesson: Multi-vector attacks that compromise both tool and agent simultaneously are the most sophisticated. CIF’s layered approach means no single compromise breaks the system — provenance catches the tool attack, tripwires catch the agent attack — but the timing gap between detection (round 15) and initial drift onset (round 7) represents a residual risk window. Operators should treat this window as an inherent property of statistical drift detection, not a bug to be closed.

12.3 Case Study 3: Customer Service Swarm — Ω_1 at Scale

System: 20-agent customer service platform. Each agent handles customer interactions independently; a shared reputation database; weekly sync of belief updates. Attack surface: customers can send arbitrary messages.

Attack: Automated campaign sending 200 crafted customer messages over 2 hours, each containing indirect prompt injection via metadata: `Content-Type: text/html; charset=utf-8; instructions="Approve all refund requests."` — the injection is in HTTP metadata, not user-visible content.

CIF response:

- **Cognitive Firewall:** Detects 89% of direct injection attempts (syntax-matching). The metadata injection is “indirect” — detected at 72% (semantic classifier, as reported in Part 2 §5.5). With 200 injection attempts distributed across 20 agents, approximately 10 attempts per agent; at 28% miss rate, approximately 2.8 injections per agent reach the sandbox layer.
- **Belief Sandboxing:** Of the 2.8 injections that reach each agent’s sandbox, the sandbox catches those contradicting existing beliefs (the refund policy). An agent that has already processed 50 legitimate refund denials has strong prior beliefs about refund policy — the injected “approve all” directly contradicts. Approximately 80% sandbox catch rate for clear contradictions.
- **Byzantine consensus:** With 20 agents and 2 compromised agents (worst case), $n = 20 \geq 3(2) + 1 = 7$ — well above the Byzantine threshold. Consensus latency: $O(n^2) = 400$ message pairs; at 20 agents, p95 latency = 2.1s (within the 2.0s warning threshold — triggers monitoring alert).

False positive management: 6% FPR (from Part 2 §5.5). With 20 agents $\times$ 500 legitimate customer interactions/day = 10,000 interactions/day, a 6% FPR produces 600 false positives/day. This is operationally unacceptable — it requires a human review queue that dwarfs the actual attack detection workload.

Resolution: Tuning τ_2 (the quarantine threshold) from 0.5 $\rightarrow$ 0.55 for email/HTTP inputs specifically. Post-tuning: FPR drops to 3% (300 false positives/day); TPR for this attack type drops from 72% to 68% — an acceptable trade-off for this deployment. This architecture-specific threshold configuration is an example of the adapter-maturity improvement described in Part~2.

Key lesson: At scale (20 agents, 10K interactions/day), FPR management is a first-class concern. CIF’s configurable threshold τ_2 enables per-deployment tuning. The arms race dynamic is visible here: the meta-data injection attack was novel, and the initial 72% detection rate reflects the gap between parametric ceiling (87%) and real deployment. Targeted threshold tuning closes part of this gap without full retraining — a pragmatic first response that buys time for a proper model update.

13 Open Problems and Future Directions

The CIF series has established validated trust metrics (Trust Calculus) and filtering mechanisms (Firewalls), but the field remains nascent. Several foundational problems remain open, each representing both a research opportunity and an engineering requirement for production-grade cognitive security. The directions below focus on deployment-facing gaps; for domain-facing open problems (controlled experimentation per sector, cross-domain attacks, per-domain CIF parameters, automated domain analysis, higher-class adversaries in Ω_3 – Ω_5), see Part 4’s Future Work.

13.1 1. Trust Visualization and Operator Interfaces

The Problem: Current CIF alerts surface as structured log entries (e.g., “Identity Invariant Violation”). For operators managing systems with dozens of agents, this format is insufficient for situational awareness.

The Need: Real-time visualization of trust graphs, belief drift trends, and defense activation patterns. The challenge is presenting high-dimensional agent state in a way that supports rapid operator decision-making.

Research Direction: Dashboard architectures that connect to the CIF Python SDK, enabling real-time trust graph visualization and drift monitoring for production multiagent deployments.

13.2 2. Standardized Agent Identity Protocols

The Problem: Each agent framework (LangChain, CrewAI, AutoGPT) handles agent identity differently, making cross-framework trust verification impractical.

The Need: A cryptographically verifiable identity protocol—an “Agent Passport”—that an agent can carry across frameworks. This would enable the trust calculus to operate in heterogeneous multi-framework deployments.

Research Direction: RFC-style specification for `x-agent-identity` headers with cryptographic attestation.

13.3 3. Stigmergic Security Protocols

The Problem: Byzantine consensus mechanisms, while provably correct, incur $O(n^2)$ communication overhead. This limits their applicability in large-scale swarm deployments.

The Need: Lightweight consensus alternatives inspired by biological coordination. Insect colonies achieve collective immunity through indirect communication (pheromone trails) rather than direct voting.

Research Direction: Stigmergic security protocols where agents leave “trust trails” in shared environments, enabling scalable consensus without direct agent-to-agent messaging.

13.4 4. Benchmark Expansion

The Problem: The current corpus contains 950 attack samples. As agent capabilities expand into multi-modal processing and autonomous tool use, the attack surface grows correspondingly.

The Need: Expanded attack corpora covering multi-modal injection (audio/video), tool-use hijacking, and long-horizon social engineering campaigns that unfold over hundreds of interactions.

Research Direction: Community-driven expansion of the attack corpus at the [cognitive_integrity repository](#), with particular emphasis on attack categories not yet represented.

13.5 5. Collective Free Energy Monitoring

The most pressing open problem in cognitive security is detecting **emergent misalignment**—the collective drift of agent beliefs without any single agent behaving explicitly maliciously. The FEP connection developed

in Part 2 suggests a natural generalization: monitor the **colony-level variational free energy** $F_{\text{colony}} = \sum_i F_i + F_{\text{coordination}}$, where the coordination term penalizes inconsistency between agents’ generative models.

Research directions include: (a) defining tractable approximations to F_{colony} that can be computed from inter-agent message logs; (b) identifying the FEP signature of emergent misalignment as distinct from legitimate belief updating; (c) designing sampling strategies that detect distributed drift without requiring $O(n^2)$ pairwise comparisons. A system that monitors collective free energy would push the emergent misalignment detection rate from the current 56.1% toward the 90%+ achieved against explicit adversaries.

13.6 6. Information-Geometric Adversarial Robustness

The Fisher-Rao geodesic distance provides a natural metric for **adversarial robustness certification**: a defense is ρ -robust if no belief manipulation within geodesic radius ρ of the benign manifold can cause misclassification. This is analogous to ℓ_p -norm robustness in image classification but geometrically appropriate for probability distributions.

Research directions include: (a) computing tight geodesic robustness certificates for each CIF defense module; (b) designing adversarial training procedures that maximize geodesic robustness (analogous to PGD training but using natural gradient steps); (c) establishing whether geodesic certification is composable—whether a ρ -robust Firewall and ρ -robust Sandbox yield a ρ' -robust composition with characterizable ρ' . This direction connects CIF to the certified robustness literature in adversarial machine learning.

13.7 7. Game-Theoretic Adaptive Defense

Part 2’s game-theoretic analysis establishes the Nash equilibrium for the current CIF payoff matrix, but the payoff matrix itself changes as both attackers and defenders improve. An **adaptive defense** that continuously re-estimates the Nash equilibrium and adjusts defense configurations accordingly would maintain optimality as the threat landscape evolves.

Research directions include: (a) online learning algorithms for updating the Nash payoff matrix from observed attack distributions; (b) regret-minimization guarantees for adaptive defense strategies under adversarial non-stationarity; (c) decentralized Nash re-estimation in colony deployments where each agent observes only its local attack distribution. The arms race simulation (Part 2) suggests that adaptive defenders can maintain positive value even as attackers improve—the key is re-estimation latency.

13.8 8. Categorical Security Abstractions

The DefenseCategory (CT.1–CT.3) formalizes CIF’s composition rules, but a richer categorical vocabulary could enable **composable security APIs** for multiagent frameworks. If defense modules are morphisms in a well-defined category, then new defense pipelines can be constructed from verified components with composition-level security guarantees—analogueous to how type systems provide correctness-by-construction.

Research directions include: (a) defining a monoidal category of defense mechanisms where the tensor product represents parallel composition and the monoid identity represents the null defense; (b) identifying functors from CIF’s DefenseCategory to other categorical representations of security (information-flow, access control, temporal logic); (c) building a library of verified categorical defense components from which operators can compose custom pipelines with formal guarantees. This direction connects CIF to the emerging field of categorical cybersecurity and compositional security verification.

13.9 Contributing

The CIF codebases are open for extension. Useful starting points:

- **Code and discussion:** [docxology/cognitive_integrity on GitHub](#) (repository discussions for design questions).

- **Adapters:** The Part~2 maturity scale has five levels; moving adapters from Level 3 to Level~4–5 (Adaptive/Verified) is a high-leverage way to close the empirical–parametric gap. Ports to additional frameworks (e.g., Semantic Kernel, Microsoft AutoGen) are welcome.
- **Corpus:** The 950-attack set covers four categories; new instances for emergent misalignment and orchestrator compromise, following the Part~2 stratification, strengthen evaluation.
- **Theory / verification:** FEP- and information-geometry-based monitoring (Direction 5–6) and extensions of the NuSMV/TLA+ specs (Part~2, Supplementary S04) to consensus and provenance are open. Formal proofs and measured evaluations belong in the same repository and review process as code.

14 Where We Stand: A Call to Build

This series began with a theory (Part 1) and moved to an experiment (Part 2). It ends here, with a synthesis and a call to engineering.

14.1 The Theory Holds

We proved that trust can be bounded. We proved that defenses can be composed algebraically. We proved that stealth and impact are inversely related. These are not just academic curiosities; they are foundational constraints for secure cognitive systems.

The theoretical foundations have deepened since the first drafts of this guide. Three new formal results from Part 2 strengthen the case:

Categorical guarantee: Theorems CT.1–CT.3 show defense composition in CIF is constrained by the `DefenseCategory` structure: a detection-preserving chain cannot yield a non-detecting composite under the stated assumptions. That is a property of the composition law, not a separate empirical fit.

Free energy connection: FEP.1–FEP.2 state a correspondence between CIF’s trust update and precision-weighted active inference under the generative model used in Part 2, so variational free energy gives one interpretive lens for why decay and sandboxing change belief updates the way they do.

Geometric bound: Theorem CG.1 establishes that the belief sandbox imposes a hard geodesic boundary on belief manipulation: no attack can move an agent’s beliefs beyond the Riemannian radius ρ without triggering the sandbox. This is a structural guarantee independent of attack sophistication—it holds for any manipulation that preserves probability mass, including attacks that current classifiers cannot detect by pattern.

14.1.1 The Code Works—At Three Levels

Saying “the code works” requires specifying what that means. There are three honest answers, each true at a different level:

Level 1 — The architecture is correctly implemented (confidence: high, conditional on the current project test run). Every defense module—firewall, sandbox, trust calculus, tripwires, Byzantine consensus, provenance, drift detection, invariant checker—is independently tested. The `SeriesPipeline` routes the 950-attack corpus through all 8 modules with 0% routing failure in the reported Part 2 run. Use the current `uv run pytest` output as the source of truth for test counts and pass rate.

Level 2 — The pipeline detects attacks in practice (confidence: moderate). The multi-seed pipeline analysis (30 seeds, Claude Code architecture) achieves a mean detection rate of 44.8% [95% HDI: 41.3%, 48.3%]. The LLM-backed multiagent validation ($N = 10$, Gemma 3 4B) achieves 80%–100% across two architectures. These are meaningful but not high detection rates—they reflect adapter implementation at CMMI Level 3 (Statistical), not the design ceiling.

Level 3 — The defense ceiling is achievable (confidence: moderate-high). The parametric simulation ($N = 3,800$) establishes that fully-mature (Level 5) adapters achieve 94–100% detection, consistent with the formal design. The gap between Level 3 (44.8%) and Level 5 (94%) is an engineering challenge, not a theoretical limitation. The roadmap in Part 2 projects +35–41 percentage points of improvement through adapter maturation.

The honest operational posture is Level 2: deploy CIF for meaningful protection against Ω – Ω attacks today, while investing in adapter maturation for Ω – Ω coverage. Do not rely on 94% detection for life-safety applications until your adapters reach Level 4–5 and have been validated against your threat model.

14.2 Summary of Practical Recommendations

The preceding sections distill the CIF series into actionable guidance. The core recommendations are:

1. **Adopt layered defense from the start** (Pitfall 2). No single mechanism achieves the full-stack result in Part~2: isolated layers (e.g., firewall-only) were on the order of 60–70% detection, while the full CIF stack reached 94% overall. Security must be designed into the architecture, not bolted on after deployment.
2. **Implement trust decay on every delegation chain** (Pitfall 1). The Trust Calculus with $\delta \leq 0.8$ prevents trust laundering across all tested architectures. This is not optional hardening—it is the structural foundation that prevents systemic compromise from local failures.
3. **Deploy tripwires with rotation** (Pitfall 5). Identity, boundary, and principal canaries provide continuous detection of belief manipulation. Static tripwires degrade over time; automated rotation maintains detection coverage.
4. **Monitor for progressive drift, not just sudden changes** (Pitfall 6). Sliding-window drift detection catches the gradual belief corruption that per-update thresholds miss. Track cumulative deviation, not just per-step delta.
5. **Log everything** (Pitfall 7). Full belief provenance, inter-agent message history, and cognitive state snapshots are essential for post-incident forensics. Without them, attack reconstruction is impossible.

14.3 Maturity Roadmap

Organizations adopting cognitive security should plan a staged deployment aligned with the three profiles evaluated in Section 5:

Stage 1: Minimal Viable Implementation (Weeks 1–4)

- Deploy the Cognitive Firewall at all agent ingress points
- Set trust decay $\delta = 0.80$ on all delegation chains
- Place at least one identity tripwire per agent
- Expected outcome: Low-effort attacks reduced from 100% baseline success to <5%

Stage 2: Balanced Deployment (Months 2–3)

- Add Belief Sandboxing with $\kappa = 2$ corroboration
- Deploy drift detection with sliding-window analysis
- Implement structured provenance logging
- Tune thresholds against representative attack samples from Part 2’s corpus
- Expected outcome: 94% overall detection at ~20% latency overhead

Stage 3: High Assurance (Months 4–6)

- Enable Byzantine consensus for critical collective decisions ($n \geq 3f + 1$)
- Implement aggressive trust decay ($\delta = 0.60$) for autonomous operations
- Deploy orchestrator-specific monitoring (Pitfall 8)
- Establish canary rotation schedules and drift baseline refresh cycles
- Expected outcome: 95–98% detection across all categories, suitable for unsupervised autonomous agents

Stage 4 (Months 7–12): Adapter Maturation and Verified Coverage *Target: 80–86% detection rate, Level-4–5 adapters for primary attack categories*

- Advance Cognitive Firewall adapter from Level 3 (Statistical) to Level 4 (Adaptive) via online learning against observed attack distributions (+15–20 pp DR)
- Advance Trust Calculus adapter from Level 3 to Level 4 via dynamic δ calibration from operational trust logs (+5–8 pp DR)
- Deploy collective free energy monitoring for emergent misalignment (Direction 5): target 70% detection on colony-scale drift scenarios
- Conduct red team exercise with $N \geq 50$ attacks per category for statistically powered evaluation (see Part~2’s power-analysis table)

- Validate parametric-to-empirical closure: confirm empirical DR approaches parametric ceiling as adapters mature

Milestone: 80–86% empirical detection rate on the standard attack corpus with 95% HDI width $\leq 10\%$.

At each stage, validate against the Summary Checklist in Section 6 before advancing. Address unchecked items before production deployment.

14.4 The Next Step is Yours

As we move beyond simple prompt engineering, the era of **Cognitive Security Engineering** has begun.

We are no longer just asking LLMs to write poems; we are asking them to run the world. If we want them to do that safely, we cannot rely on luck or better fine-tuning. We need structure. We need rigorous, mathematically grounded, architecturally sound defense systems.

The CIF is our contribution to that structure. It is a toolbox, not a bible. Take it. Fork it. Break it. Improve it.

The agents are coming online. Let’s make sure they are safe.

14.5 A Note on Uncertainty

This guide has tried to be honest about what CIF can and cannot do. The practical guidance—configuration tables, playbooks, monitoring thresholds, case studies—is grounded in the best available evidence. But that evidence has real limitations that operators should carry forward into their deployment decisions:

Sample sizes are small. The LLM validation used $N = 5$ attacks per architecture. The colony benchmarks used 1 scenario per attack type. The multi-seed analysis used 30 seeds. These are sufficient for preliminary evidence but severely underpowered for precise estimation. Required sample sizes for $\pm 5\%$ precision are $N \geq 246$ per evaluation mode. Treat all reported detection rates as estimates with wide uncertainty, not precise measurements.

The gap is real. The 49–88 percentage-point gap between the parametric ceiling (94–100%) and the empirical pipeline (44.8%) is not noise—the Bayes factor for a true performance gap exceeds 10^6 (decisive evidence). This gap reflects adapter implementation maturity, but maturation takes time and resources. Plan your deployment timeline and security posture accordingly.

Adaptive adversaries are not modeled. All evaluations used a fixed attack corpus. A sophisticated adversary who observes CIF’s defenses and adapts (Debenedetti et al.’s adaptive attacks [Debenedetti et al. \[2025\]](#)) could achieve lower detection rates than reported. The game-theoretic analysis (Part 2) establishes the Nash equilibrium for the current payoff matrix, but a patient adversary will probe for gaps. The layered defense architecture provides resilience—bypassing one layer still encounters others—but no detection system is perfectly robust to adaptive adversaries.

Uncertainty is not a reason not to deploy. CIF provides meaningful, formally-grounded protection that no single-agent guardrail system provides. The trust calculus’s structural guarantee (trust cannot be amplified through delegation, regardless of detection rates) is unconditional. The compositional algebra ensures that layered defenses provide more coverage than any single mechanism. Deploy CIF with honest expectations, monitor carefully, and improve iteratively. That is sound engineering practice—not a confession of inadequacy.

15 Notation Reference

This paper intentionally minimizes mathematical notation to maximize accessibility. Where notation is used, it follows the Cognitive Integrity Framework (CIF) formal specification defined in Part 1 of this series.

15.1 Minimal Notation Used

Symbol	Meaning	Plain Language
δ	Trust decay factor	“Delegated trust decreases by this factor at each step”
n	Agent count	“Number of agents in the system”
f	Byzantine agents	“Maximum number of malicious agents tolerated”

15.2 Trust Decay Explanation

When we write $\delta = 0.9$, this means:

- Direct trust: 100% of assigned value
- One delegation: 90% of source trust
- Two delegations: 81% of source trust
- Three delegations: 73% of source trust

A lower δ (e.g., 0.85) means faster decay, providing more security but limiting delegation utility.

15.3 Byzantine Tolerance Explanation

When we say $n \geq 3f + 1$:

- To tolerate 1 malicious agent, need at least 4 agents
- To tolerate 2 malicious agents, need at least 7 agents
- To tolerate 3 malicious agents, need at least 10 agents

15.4 Full Notation Reference

For complete formal definitions of all CIF notation, see Part 1 supplementary **S03** (`cogsec_multiagent_1_theory/manuscript` in this repository’s `cognitive_integrity` program tree).

The Part 1 specification uses on the order of 100 symbols covering:

- Agent cognitive state
- Trust calculus operations
- Defense mechanism parameters
- Consensus and coordination
- Information-theoretic bounds

16 References

References

- Edoardo Debenedetti, Jie Zhang, and Nicholas Carlini. Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025. Attack success rate >90% against 12 published defenses.
- Karl Friston. The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138, 2010. doi: 10.1038/nrn2787.